Submodule 33.09: Optimal Cluster Selection: Elbow Method & Silhouette Score

Topic Overview & Business Search Space

Determining the optimal number of clusters (K) is a fundamental trade-off between mathematical compactness and business operability. In e-commerce retail marketing:

- Segmenting customers into $K = 20$ or $K = 30$ creates hyper-fragmented micro-niches that marketing departments cannot operationally execute.
- We restrict our hyperparameter search space to $K \in [1, 10]$ to discover 3 to 6 distinct, actionable personas.

---

1. Mathematical Metrics for Choosing K

A. Within-Cluster Sum of Squares (WCSS / Inertia)

Measures the internal compactness of clusters:

$$\text{WCSS} = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

As K increases, WCSS monotonically decreases. The optimal K lies at the "elbow" point where diminishing returns set in.

B. Automated Elbow Detection via `KneeLocator`

Rather than subjectively eyeballing the elbow curve, we deploy the `kneed` algorithm:

- Detects the point of maximum curvature on a convex, decreasing curve:

$$\text{knee} = \arg\max_k \left(\text{WCSS}_{\text{normalized}}(k) - \text{Line}(k) \right)$$

C. Silhouette Analysis

Measures both cluster cohesion $a(i)$ (mean intra-cluster distance) and cluster separation $b(i)$ (mean nearest-cluster distance):

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

The global silhouette score is the average $s(i)$ across all N data points.

---

2. Python Implementation

Reference: smartcart.ipynb (Cells 46 - 53).

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from kneed import KneeLocator
import matplotlib.pyplot as plt

# 1. Compute WCSS for K in [1, 10]
wcss = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)

# Automatic Elbow Detection
knee = KneeLocator(range(1, 11), wcss, curve="convex", direction="decreasing")
optimal_k = knee.elbow
print("Optimal K from KneeLocator:", optimal_k) # Output: 4

# 2. Compute Silhouette Score for K in [2, 10]
scores = []
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X_pca)
    scores.append(silhouette_score(X_pca, labels))

# 3. Dual-Axis Visualization (twinx)
k_range = range(2, 11)
fig, ax1 = plt.subplots(figsize=(8, 6))

# Primary axis: WCSS (Blue)
ax1.plot(k_range, wcss[1:], marker="o", color="blue", label="WCSS")
ax1.set_xlabel("Number of Clusters (K)")
ax1.set_ylabel("WCSS (Inertia)", color="blue")

# Secondary axis: Silhouette Score (Red Dashed)
```

```
ax2 = ax1.twinx()
ax2.plot(k_range, scores, marker="x", color="red", linestyle="--", label="Silhouet
ax2.set_ylabel("Silhouette Score", color="red")

plt.title("Optimal K Selection: Elbow Method & Silhouette Score")
plt.show()
```

---

3. Convergence on $K = 4$

1. **Elbow Analysis**: WCSS drops steeply from $K = 1$ through $K = 4$, after which the curve flattens significantly. `KneeLocator` algorithmically confirms $K^* = 4$.
2. **Silhouette Trade-off**: While higher K values yield marginal silhouette variations, $K = 4$ maintains a solid score while preserving broad, commercially viable segments.
3. **Conclusion**: $K = 4$ is adopted as the optimal segmentation parameter for both K-Means and Agglomerative Clustering.